Red Hat OpenShift AI Self-Managed 3.0

Working in your data science IDE

Working in your data science IDE from Red Hat OpenShift AI Self-Managed

Last Updated: 2025-12-01

Red Hat OpenShift AI Self-Managed 3.0 Working in your data science IDE

Working in your data science IDE from Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

Prepare your data science integrated development environment (IDE) for developing machine learning models.

Table of Contents

PREFACE

In Red Hat OpenShift AI, when you create a workbench, you select a workbench image that includes an integrated development environment (IDE) for developing your machine learning (ML) models.

You can use the following data science IDEs for developing ML models with OpenShift AI:

- JupyterLab
- code-server
- RStudio Server (Technology Preview feature)

NOTE

The RStudio workbench images are currently unavailable for disconnected environments.

For information about RStudio Server, see the Release Notes.

CHAPTER 1. ACCESSING YOUR WORKBENCH IDE

To access a workbench IDE, use the link provided in the OpenShift AI interface.

Prerequisite

- You have created a project and a workbench.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
2. Click the name of the project that contains the workbench.
3. Click the **Workbenches** tab.
4. If the status of the workbench is **Running**, skip to the next step.
   If the status of the workbench is **Stopped**, in the **Status** column for the workbench, click **Start**.

   The **Status** column changes from **Stopped** to **Starting** when the workbench server is starting, and then to **Running** when the workbench has successfully started.
5. Click the open icon () next to the workbench.

Verification

- A new browser window opens for the workbench IDE.

CHAPTER 2. WORKING IN JUPYTERLAB

JupyterLab is a web-based interactive development environment for Jupyter notebooks, code, and data. You can configure and arrange workflows in data science and machine learning. JupyterLab is an open source web application that supports over 40 programming languages, including Python and R.

2.1. CREATING AND IMPORTING JUPYTER NOTEBOOKS

You can create a blank Jupyter notebook or import a Jupyter notebook in JupyterLab from several different sources.

2.1.1. Creating a Jupyter notebook

You can create a Jupyter notebook from an existing notebook container image to access its resources and properties. The **Workbench control panel** contains a list of available container images that you can run as a single-user workbench.

Prerequisites

- Ensure that you have logged in to Red Hat OpenShift AI.
- Ensure that you have launched your workbench and logged in to JupyterLab.
- The workbench image exists in a registry, image stream, and is accessible.

Procedure

1. Click **File → New → Notebook**.
2. If prompted, select a kernel for your Jupyter notebook from the list.
   If you want to use a kernel, click **Select**. If you do not want to use a kernel, click **No Kernel**.

Verification

- Check that the notebook file is visible in the JupyterLab interface.

2.1.2. Uploading an existing notebook file to JupyterLab from local storage

You can load an existing notebook file from local storage into JupyterLab to continue work, or adapt a project for a new use case.

Prerequisites

- Credentials for logging in to JupyterLab.
- You have a launched and running workbench based on a JupyterLab image.
- A notebook file exists in your local storage.

Procedure

1. In the **File Browser** in the left sidebar of the JupyterLab interface, click **Upload Files** ().

2. Locate and select the notebook file and then click **Open**.
   The file is displayed in the **File Browser**.

Verification

- The notebook file is displayed in the **File Browser** in the left sidebar of the JupyterLab interface.
- You can open the notebook file in JupyterLab.

2.1.3. Additional resources

- Collaborating on Jupyter notebooks by using Git

2.2. COLLABORATING ON JUPYTER NOTEBOOKS BY USING GIT

If your files are stored in Git version control, you can clone a Git repository to work with them in JupyterLab. When you are ready, you can push your changes back to the Git repository so that others can review or use your models.

2.2.1. Uploading an existing notebook file from a Git repository by using JupyterLab

You can use the JupyterLab user interface to clone a Git repository into your workspace to continue your work or integrate files from an external project.

Prerequisites

- You have a launched and running workbench based on a JupyterLab image.
- Read access for the Git repository you want to clone.

Procedure

1. Copy the HTTPS URL for the Git repository.
   - In GitHub, click **Code → HTTPS** and then click the **Copy URL to clipboard** icon.
   - In GitLab, click **Code** and then click the **Copy URL** icon under **Clone with HTTPS**.
2. In the JupyterLab interface, click the **Git Clone** button ( ).
   You can also click **Git → Clone a repository** in the menu, or click the Git icon ( ) and click the **Clone a repository** button.
   The **Clone a repo** dialog opens.
3. Enter the HTTPS URL of the repository that contains your notebook file.
4. Click **CLONE**.
5. If prompted, enter your username and password for the Git repository.

Verification

- Check that the contents of the repository are visible in the file browser in JupyterLab, or run the **ls** command in the terminal to verify that the repository shows as a directory.

2.2.2. Uploading an existing notebook file to JupyterLab from a Git repository by using the CLI

You can use the command line interface to clone a Git repository into your workspace to continue your work or integrate files from an external project.

Prerequisites

- You have a launched and running workbench based on a JupyterLab image.

Procedure

1. Copy the HTTPS URL for the Git repository.
   - In GitHub, click **Code → HTTPS** and then click the **Copy URL to clipboard** icon.
   - In GitLab, click **Code** and then click the **Copy URL** icon under **Clone with HTTPS**.
2. In JupyterLab, click **File → New → Terminal** to open a terminal window.
3. Enter the **git clone** command:

   ```
   git clone <git-clone-URL>
   ```

   Replace **git-clone-URL>** with the HTTPS URL, for example:

   ```
   [1234567890@jupyter-nb-jdoe ~]$ git clone https://github.com/example/myrepo.git
   Cloning into myrepo...
   remote: Enumerating objects: 11, done.
   remote: Counting objects: 100% (11/11), done.
   remote: Compressing objects: 100% (10/10), done.
   remote: Total 2821 (delta 1), reused 5 (delta 1), pack-reused 2810
   Receiving objects: 100% (2821/2821), 39.17 MiB | 23.89 MiB/s, done.
   Resolving deltas: 100% (1416/1416), done.
   ```

Verification

- Check that the contents of the repository are visible in the file browser in JupyterLab, or run the **ls** command in the terminal to verify that the repository shows as a directory.

2.2.3. Updating your project with changes from a remote Git repository

You can pull changes made by other users into your project from a remote Git repository.

Prerequisites

- You have a launched and running workbench based on a JupyterLab image.
- You have credentials for logging in to Jupyter.
- You have configured the remote Git repository.

- You have permissions to pull files from the remote Git repository to your local repository.
- You have imported the Git repository into JupyterLab, and the contents of the repository are visible in the file browser in JupyterLab.

Procedure

1. In the JupyterLab interface, click the **Git** button ().
2. Click the **Pull latest changes** button ().

Verification

- You can view the changes pulled from the remote repository on the **History** tab in the Git pane.

2.2.4. Pushing project changes to a Git repository

To build and deploy your application in a production environment, upload your work to a remote Git repository.

Prerequisites

- You have opened a Jupyter notebook in the JupyterLab interface.
- You have added the relevant Git repository to your workbench.
- You have permission to push changes to the relevant Git repository.
- You have installed the Git version control extension.

Procedure

1. Click **File → Save All** to save any unsaved changes.
2. Click the Git icon () to open the Git pane in the JupyterLab interface.
3. Confirm that your changed files appear under **Changed**.
   If your changed files appear under **Untracked**, click **Git → Simple Staging** to enable a simplified Git process.
4. Commit your changes.
   a. Ensure that all files under **Changed** have a blue checkmark beside them.
   b. In the **Summary** field, enter a brief description of the changes you made.
   c. Click **Commit**.
5. Click **Git → Push to Remote** to push your changes to the remote repository.
6. When prompted, enter your Git credentials and click **OK**.

Verification

- Your most recently pushed changes are visible in the remote Git repository.

2.3. MANAGING PYTHON PACKAGES

In JupyterLab, you can view the Python packages that are installed on your workbench image and install additional packages.

2.3.1. Viewing Python packages installed on your workbench

You can check which Python packages are installed on your workbench and which version of the package you have by running the **pip** tool in a notebook cell.

Prerequisites

- Log in to JupyterLab and open a Jupyter notebook.

Procedure

1. Enter the following in a new cell in your Jupyter notebook:

```
!pip list
```

2. Run the cell.

Verification

- The output shows an alphabetical list of all installed Python packages and their versions. For example, if you use the **pip list** command immediately after creating a workbench that uses the **Minimal** image, the first packages shown are similar to the following:

```
Package                           Version
--------------------------------- ----------
aiohttp                            3.7.3
alembic                             1.5.2
appdirs                            1.4.4
argo-workflows                        3.6.1
argon2-cffi                        20.1.0
async-generator                       1.10
async-timeout                        3.0.1
attrdict                          2.0.1
attrs                             20.3.0
backcall                            0.2.0
```

Additional resources

- Installing Python packages on your workbench

2.3.2. Installing Python packages on your workbench

You can install Python packages that are not part of the default workbench by adding the package and the version to a **requirements.txt** file and then running the **pip install** command in a notebook cell.

NOTE

Although you can install packages directly, it is recommended that you use a **requirements.txt** file so that the packages stated in the file can be easily re-used across different workbenches.

Prerequisites

- Log in to JupyterLab and open a Jupyter notebook.

Procedure

1. Create a new text file using one of the following methods:
   - Click **+** to open a new launcher and then click **Text file**.
   - Click **File → New → Text File**.
2. Rename the text file to **requirements.txt**.
   a. Right-click the name of the file and then click **Rename Text**. The **Rename File** dialog opens.
   b. Enter **requirements.txt** in the **New Name** field and then click **Rename**.
3. Add the packages to install to the **requirements.txt** file.

```
altair
```

   You can specify the exact version to install by using the **==** (equal to) operator, for example:

```
altair==4.1.0
```

   NOTE

   Red Hat recommends specifying exact package versions to enhance the stability of your workbench over time. New package versions can introduce undesirable or unexpected changes in your environment's behavior.

   To install multiple packages at the same time, place each package on a separate line.
4. Install the packages in **requirements.txt** to your server by using a notebook cell.
   a. Create a new notebook cell and enter the following command:

```
!pip install -r requirements.txt
```

   b. Run the cell by pressing Shift and Enter.

IMPORTANT

The **pip install** command installs the package on your workbench. However, you must run the **import** statement in a code cell to use the package in your code.

```
import altair
```

Verification

- Confirm that the packages in the **requirements.txt** file appear in the list of packages installed on the workbench. See Viewing Python packages installed on your workbench for details.

2.4. TROUBLESHOOTING COMMON PROBLEMS IN WORKBENCHES FOR USERS

If you are seeing errors in Red Hat OpenShift AI related to Jupyter, your Jupyter notebooks, or your workbench, read this section to understand what could be causing the problem.

If you cannot see your problem here or in the release notes, contact Red Hat Support.

I see a 403: Forbidden error when I log in to Jupyter

Problem

If your cluster administrator has configured OpenShift AI user groups, your username might not be added to the default user group or the default administrator group for OpenShift AI.

Resolution

```
Contact your cluster administrator so that they can add you to the correct group/s.
```

My workbench does not start

Problem

The OpenShift cluster that hosts your workbench might not have access to enough resources, or the workbench pod may have failed.

Resolution

Check the logs in the **Events** section in OpenShift for error messages associated with the problem. For example:

```
Server requested
2021-10-28T13:31:29.830991Z [Warning] 0/7 nodes are available: 2 Insufficient memory,
2 node(s) had taint {node-role.kubernetes.io/infra: }, that the pod didn't tolerate, 3 node(s) had taint
{node-role.kubernetes.io/master: },
that the pod didn't tolerate.
```

Contact your cluster administrator with details of any relevant error messages so that they can perform further checks.

I see a database or disk is full error or a no space left on device error when I run my notebook cells

Problem

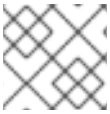
You might have run out of storage space on your workbench.

Resolution

Contact your cluster administrator so that they can perform further checks.

CHAPTER 3. WORKING IN CODE-SERVER

Code-server is a web-based interactive development environment supporting multiple programming languages, including Python, for working with Jupyter notebooks. With the code-server workbench image, you can customize your workbench environment to meet your needs using a variety of extensions to add new languages, themes, debuggers, and connect to additional services. For more information, see code-server in GitHub .



NOTE

Elyra-based pipelines are not available with the code-server workbench image.

3.1. CREATING CODE-SERVER WORKBENCHES

You can create a blank Jupyter notebook or import a Jupyter notebook in code-server from several different sources.

3.1.1. Creating a workbench

When you create a workbench, you specify an image (an IDE, packages, and other dependencies). You can also configure connections, cluster storage, and add container storage.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.
- If you created a Simple Storage Service (S3) account outside of Red Hat OpenShift AI and you want to create connections to your existing S3 storage buckets, you have the following credential information for the storage buckets:
  - Endpoint URL
  - Access key
  - Secret key
  - Region
  - Bucket name

  For more information, see Working with data in an S3-compatible object store .

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to add the workbench to.
   A project details page opens.
3. Click the **Workbenches** tab.
4. Click **Create workbench**.

The **Create workbench** page opens.

5. In the **Name** field, enter a unique name for your workbench.

6. Optional: If you want to change the default resource name for your workbench, click **Edit resource name**.
   The resource name is what your resource is labeled in OpenShift. Valid characters include lowercase letters, numbers, and hyphens (-). The resource name cannot exceed 30 characters, and it must start with a letter and end with a letter or number.

   Note: You cannot change the resource name after the workbench is created. You can edit only the display name and the description.

7. Optional: In the **Description** field, enter a description for your workbench.

8. In the **Workbench image** section, complete the fields to specify the workbench image to use with your workbench.
   From the **Image selection** list, select a workbench image that suits your use case. A workbench image includes an IDE and Python packages (reusable code). If project-scoped images exist, the **Image selection** list includes subheadings to distinguish between global images and project-scoped images.

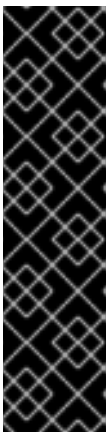
   Optionally, click **View package information** to view a list of packages that are included in the image that you selected.

   If the workbench image has multiple versions available, select the workbench image version to use from the **Version selection** list. To use the latest package versions, Red Hat recommends that you use the most recently added image.

NOTE

You can change the workbench image after you create the workbench.

9. In the **Deployment size** section, select one of the following options, depending on whether the hardware profiles feature is enabled.



IMPORTANT

The hardware profiles feature is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

- If the hardware profiles feature is not enabled:

  a. From the **Container size** list, select the appropriate size for the size of the model that you want to train or tune.
     For example, to run the example fine-tuning job described in Fine-tuning a model by using Kubeflow Training, select **Medium**.

b. From the **Hardware profile** list, select a suitable hardware profile for your workbench.
If project-scoped hardware profiles exist, the **Hardware profile** list includes subheadings to distinguish between global hardware profiles and project-scoped hardware profiles.

The hardware profile specifies the number of CPUs and the amount of memory allocated to the container, setting the guaranteed minimum (request) and maximum (limit) for both.

c. If you want to change the default values, click **Customize resource requests and limit** and enter new minimum (request) and maximum (limit) values.

10. Optional: In the **Environment variables** section, select and specify values for any environment variables.
Setting environment variables during the workbench configuration helps you save time later because you do not need to define them in the body of your workbenches, or with the IDE command line interface.

If you are using S3-compatible storage, add these recommended environment variables:

- **AWS_ACCESS_KEY_ID** specifies your Access Key ID for Amazon Web Services.
- **AWS_SECRET_ACCESS_KEY** specifies your Secret access key for the account specified in **AWS_ACCESS_KEY_ID**.

OpenShift AI stores the credentials as Kubernetes secrets in a protected namespace if you select **Secret** when you add the variable.

11. In the **Cluster storage** section, configure the storage for your workbench. Select one of the following options:

- **Create new persistent storage** to create storage that is retained after you shut down your workbench. Complete the relevant fields to define the storage:

a. Enter a **name** for the cluster storage.

b. Enter a **description** for the cluster storage.

c. Select a **storage class** for the cluster storage.

NOTE

You cannot change the storage class after you add the cluster storage to the workbench.

d. For storage classes that support multiple access modes, select an **Access mode** to define how the volume can be accessed. For more information, see About persistent storage.
Only the access modes that have been enabled for the storage class by your cluster and OpenShift AI administrators are visible.

e. Under **Persistent storage size**, enter a new size in gibibytes or mebibytes.

- **Use existing persistent storage** to reuse existing storage and select the storage from the **Persistent storage** list.

12. Optional: You can add a connection to your workbench. A connection is a resource that contains the configuration parameters needed to connect to a data source or sink, such as an object storage bucket. You can use storage buckets for storing data, models, and pipeline artifacts. You can also use a connection to specify the location of a model that you want to deploy. In the **Connections** section, use an existing connection or create a new connection:

    - Use an existing connection as follows:

        a. Click **Attach existing connections**.

        b. From the **Connection** list, select a connection that you previously defined.

    - Create a new connection as follows:

        a. Click **Create connection**. The **Add connection** dialog opens.

        b. From the **Connection type** drop-down list, select the type of connection. The **Connection details** section is displayed.

        c. If you selected **S3 compatible object storage** in the preceding step, configure the connection details:

            i. In the **Connection name** field, enter a unique name for the connection.

            ii. Optional: In the **Description** field, enter a description for the connection.

            iii. In the **Access key** field, enter the access key ID for the S3-compatible object storage provider.

            iv. In the **Secret key** field, enter the secret access key for the S3-compatible object storage account that you specified.

            v. In the **Endpoint** field, enter the endpoint of your S3-compatible object storage bucket.

            vi. In the **Region** field, enter the default region of your S3-compatible object storage account.

            vii. In the **Bucket** field, enter the name of your S3-compatible object storage bucket.

            viii. Click **Create**.

        d. If you selected **URI** in the preceding step, configure the connection details:

            i. In the **Connection name** field, enter a unique name for the connection.

            ii. Optional: In the **Description** field, enter a description for the connection.

            iii. In the **URI** field, enter the Uniform Resource Identifier (URI).

            iv. Click **Create**.

13. Click **Create workbench**.

Verification

- The workbench that you created is visible on the **Workbenches** tab for the project.

- Any cluster storage that you associated with the workbench during the creation process is displayed on the **Cluster storage** tab for the project.
- The **Status** column on the **Workbenches** tab displays a status of **Starting** when the workbench server is starting, and **Running** when the workbench has successfully started.
- Optional: Click the open icon () to open the IDE in a new window.

3.1.2. Uploading an existing notebook file to code-server from local storage

You can load an existing notebook file from local storage into code-server to continue work, or adapt a project for a new use case.

Prerequisites

- You have a running code-server workbench.
- You have a notebook file in your local storage.

Procedure

1. In your code-server window, from the Activity Bar, select the menu icon () → **File** → **Open File**.
2. In the **Open File** dialog, click the **Show Local** button.
3. Locate and select the notebook file and then click **Open**.
   The file is displayed in the code-server window.
4. Save the file and then push the changes to your repository.

Verification

- The notebook file is displayed in the code-server Explorer view.
- You can open the notebook file in the code-server window.

3.2. COLLABORATING ON WORKBENCHES IN CODE-SERVER BY USING GIT

If your files are stored in Git version control, you can clone a Git repository to work with them in code-server. When you are ready, you can push your changes back to the Git repository so that others can review or use your models.

3.2.1. Uploading an existing notebook file from a Git repository by using code-server

You can use the code-server user interface to clone a Git repository into your workspace to continue your work or integrate files from an external project.

Prerequisites

- You have a running code-server workbench.

- You have read access for the Git repository you want to clone.

Procedure

1. Copy the HTTPS URL for the Git repository.
   - In GitHub, click **Code → HTTPS** and then click the **Copy URL to clipboard** icon.
   - In GitLab, click **Code** and then click the **Copy URL** icon under **Clone with HTTPS**.
2. In your code-server window, from the Activity Bar, select the menu icon ( ) → **View → Command Palette**.
3. In the Command Palette, enter **Git: Clone**, and then select **Git: Clone** from the list.
4. Paste the HTTPS URL of the repository that contains your notebook file, and then press Enter.
5. If prompted, enter your username and password for the Git repository.
6. Select a folder to clone the repository into, and then click **OK**.
7. When the repository is cloned, a dialog opens asking if you want to open the cloned repository. Click **Open** in the dialog.

Verification

- Check that the contents of the repository are visible in the code-server Explorer view, or run the **ls** command in the terminal to verify that the repository shows as a directory.

3.2.2. Uploading an existing notebook file to code-server from a Git repository by using the CLI

You can use the command line interface to clone a Git repository into your workspace to continue your work or integrate files from an external project.

Prerequisites

- You have a running code-server workbench.

Procedure

1. Copy the HTTPS URL for the Git repository.
   - In GitHub, click **Code → HTTPS** and then click the **Copy URL to clipboard** icon.
   - In GitLab, click **Code** and then click the **Copy URL** icon under **Clone with HTTPS**.
2. In your code-server window, from the Activity Bar, select the menu icon ( ) → **Terminal → New Terminal** to open a terminal window.
3. Enter the **git clone** command:

```
git clone <git-clone-URL>
```

Replace **<git-clone-URL>** with the HTTPS URL, for example:

```
$ git clone https://github.com/example/myrepo.git
Cloning into myrepo...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 2821 (delta 1), reused 5 (delta 1), pack-reused 2810
Receiving objects: 100% (2821/2821), 39.17 MiB | 23.89 MiB/s, done.
Resolving deltas: 100% (1416/1416), done.
```

Verification

- Check that the contents of the repository are visible in the code-server Explorer view, or run the **ls** command in the terminal to verify that the repository shows as a directory.

3.2.3. Updating your project in code-server with changes from a remote Git repository

You can pull changes made by other users into your workbench from a remote Git repository.

Prerequisites

- You have configured the remote Git repository.
- You have imported the Git repository into code-server, and the contents of the repository are visible in the Explorer view in code-server.
- You have permissions to pull files from the remote Git repository to your local repository.
- You have a running code-server workbench.

Procedure

1. In your code-server window, from the Activity Bar, click the **Source Control** icon ().
2. Click the **Views and More Actions** button (**...**), and then select **Pull**.

Verification

- You can view the changes pulled from the remote repository in the **Source Control** pane.

3.2.4. Pushing project changes in code-server to a Git repository

To build and deploy your application in a production environment, upload your work to a remote Git repository.

Prerequisites

- You have a running code-server workbench.
- You have added the relevant Git repository in code-server.

- You have permission to push changes to the relevant Git repository.
- You have installed the Git version control extension.

Procedure

1. In your code-server window, from the Activity Bar, select the menu icon () → **File** → **Save All** to save any unsaved changes.
2. Click the **Source Control** icon () to open the Source Control pane.
3. Confirm that your changed files appear under **Changes**.
4. Next to the **Changes** heading, click the **Stage All Changes** button (+). The staged files move to the **Staged Changes** section.
5. In the **Message** field, enter a brief description of the changes you made.
6. Next to the **Commit** button, click the **More Actions...** button, and then click **Commit & Sync**.
7. If prompted, enter your Git credentials and click **OK**.

Verification

- Your most recently pushed changes are visible in the remote Git repository.

3.3. MANAGING PYTHON PACKAGES IN CODE-SERVER

In code-server, you can view the Python packages that are installed on your workbench image and install additional packages.

3.3.1. Viewing Python packages installed on your code-server workbench

You can check which Python packages are installed on your workbench and which version of the package you have by running the **pip** tool in a terminal window.

Prerequisites

- You have a running code-server workbench.

Procedure

1. In your code-server window, from the Activity Bar, select the menu icon () → **Terminal** → **New Terminal** to open a terminal window.
2. Enter the **pip list** command.

```
pip list
```

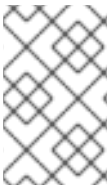
Verification

- The output shows an alphabetical list of all installed Python packages and their versions. For example, if you use the **pip list** command immediately after creating a workbench that uses the **Minimal** image, the first packages shown are similar to the following:

```
Package                Version
---------------------- ----------
asttokens               2.4.1
boto3                  1.34.162
botocore               1.34.162
cachetools              5.5.0
certifi              2024.8.30
charset-normalizer       3.4.0
comm                    0.2.2
contourpy               1.3.0
cycler                 0.12.1
debugpy                 1.8.7
```

3.3.2. Installing Python packages on your code-server workbench

You can install Python packages that are not part of the default workbench image by adding the package and the version to a **requirements.txt** file and then running the **pip install** command in a terminal window.



NOTE

Although you can install packages directly, it is recommended that you use a **requirements.txt** file so that the packages stated in the file can be easily re-used across different workbenches.

Prerequisites

- You have a running code-server workbench.

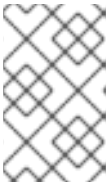
Procedure

1. In your code-server window, from the Activity Bar, select the menu icon () → **File** → **New Text File** to create a new text file.
2. Add the packages to install to the text file.

```
altair
```

You can specify the exact version to install by using the **==** (equal to) operator, for example:

```
altair==4.1.0
```



NOTE

Red Hat recommends specifying exact package versions to enhance the stability of your workbench over time. New package versions can introduce undesirable or unexpected changes in your environment's behavior.

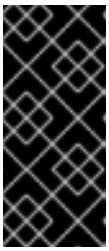
To install multiple packages at the same time, place each package on a separate line.

3. Save the text file as **requirements.txt**.

4. From the Activity Bar, select the menu icon ( ) → **Terminal** → **New Terminal** to open a terminal window.

5. Install the packages in **requirements.txt** to your server by using the following command:

```
pip install -r requirements.txt
```



IMPORTANT

The **pip install** command installs the package on your workbench. However, you must run the **import** statement to use the package in your code.

```
import altair
```

Verification

- Confirm that the packages in the **requirements.txt** file appear in the list of packages installed on the workbench. See Viewing Python packages installed on your code-server workbench for details.

3.4. INSTALLING EXTENSIONS WITH CODE-SERVER

With the code-server workbench image, you can customize your code-server environment by using extensions to add new languages, themes, and debuggers, and to connect to additional services. You can also enhance the efficiency of your data science work with extensions for syntax highlighting, auto-indentation, and bracket matching.

For details about the third-party extensions that you can install with code-server, see the Open VSX Registry.

Prerequisites

- You are logged in to Red Hat OpenShift AI.
- You have created a project that has a code-server workbench.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.

2. Click the name of the project containing the code-server workbench you want to start.
   A project details page opens.

3. Click the **Workbenches** tab.

4. If the status of the workbench that you want to use is **Running**, skip to the next step.
   If the status of the workbench is **Stopped**, in the **Status** column for the workbench, click **Start**.

The **Status** column changes from **Stopped** to **Starting** when the workbench server is starting, and then to **Running** when the workbench has successfully started.

5. Click the open icon () next to the workbench.
   The code-server window opens.

6. In the Activity Bar, click the **Extensions** icon ().

7. Search for the name of the extension you want to install.

8. Click **Install** to add the extension to your code-server environment.

Verification

- In the **Browser - Installed** list on the **Extensions** panel, confirm that the extension you installed is listed.